Student Dropout Prediction

MATH 4322 - Group 4 Final Report

Sriharsha Challangi, Kelvin Mai, Thanh Trung Do, Leah Padilla, Tien Hoang

Spring 2026

Contents

1 Introduction **2**

1.1 Data Description (Thanh Trung Do, Leah Padilla) 2

1.2 Research Question 2

2 Methods **3**

2.1 Logistic Regression (Thanh Trung Do, Leah Padilla) 3

2.2 Random Forest (Sriharsha Challangi, Kelvin Mai, Tien Hoang) 6

3 Results **10**

3.1 Logistic Regression 10

3.2 Random Forest 16

4 Conclusion **18**

4.1 Logistic regression 19

4.2 Random forest 19

4.3 Model Comparison 19

5 References **20**

6 Appendix: Full R Source Code **21**

1 Introduction

1.1 Data Description (Thanh Trung Do, Leah Padilla)

The dataset is the **Student Dropout Prediction Dataset** from Kaggle (Ali, 2024), containing 10,000 student records with 18 predictor variables and 1 binary response variable.

Response variable:

- `Dropout`: Binary (0 = retained, 1 = dropped out)

Predictor variables (inputs):

- *Demographic*: Age, Gender (Male/Female), Family_Income, Parental_Education (None, High School, Bachelor, Master, PhD)
- *Academic*: GPA, Semester_GPA, CGPA, Semester (Year 1–4), Department (Science, Arts, Business, CS, Engineering)
- *Behavioral*: Study_Hours_per_Day, Attendance_Rate, Assignment_Delay_Days, Travel_Time_Minutes, Stress_Index (1–10)
- *Socioeconomic*: Internet_Access (Yes/No), Part_Time_Job (Yes/No), Scholarship (Yes/No)

Three variables (`Family_Income`, `Study_Hours_per_Day`, `Stress_Index`) have 500 missing values each (5%), which we imputed with their respective medians.

1.2 Research Question

Which factors affect whether a student drops out, and can we accurately predict student dropout?

The response variable is `Dropout` (binary: 0 or 1), making this a classification problem.

2 Methods

2.1 Logistic Regression (Thanh Trung Do, Leah Padilla)

2.1.1 Why Logistic Regression?

Logistic regression is appropriate because our response variable is binary (Dropout = 0 or 1). As discussed in Lecture 8, logistic regression models the probability that Y belongs to one of two categories, and the output is always between 0 and 1 (unlike linear regression, which can predict values outside this range).

Advantages:

- Provides interpretable coefficients: each $\hat{\beta}_j$ tells us the change in log-odds per unit increase in X_j.
- Outputs probabilities, allowing flexible threshold selection.
- Computationally efficient and well-suited for inference (significance testing of predictors).

Disadvantages:

- Assumes a linear relationship between predictors and log-odds, which may not capture complex non-linear patterns.
- Can be unstable when classes are well-separated (Lecture 10).
- May underperform compared to tree-based methods when interactions between predictors are important.

2.1.2 Model Formula

The logistic regression model is:

$$\log\left(\frac{P(\text{Dropout} = 1 \mid X)}{1 - P(\text{Dropout} = 1 \mid X)}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

where the left-hand side is the log-odds (logit), and $X_1, \ldots, X_p$ are the 18 predictor variables. The predicted probability is:

$$P(\text{Dropout} = 1 \mid X) = \frac{\exp(\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p)}{1 + \exp(\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p)}$$

We estimate $\beta_0, \beta_1, \ldots, \beta_p$ using maximum likelihood. In R: `glm(Dropout ~ ., family = binomial)`.

2.1.3 Thought Process for Model Fitting

After fitting the full model with all 18 predictors, we examine the p-values from the `summary()` output. Predictors with large p-values are not statistically significant and may be excluded. We also use **stepwise selection (AIC)** to find a more parsimonious model and compare it against the full model.

2.1.4 Data Preprocessing

```r
library(ggplot2)
library(dplyr)
library(corrplot)

data <- read.csv("student_dropout_dataset_v3.csv")

# handling missing data
data$Family_Income[is.na(data$Family_Income)] <- median(data$Family_Income, na.rm = TRUE)
data$Study_Hours_per_Day[is.na(data$Study_Hours_per_Day)] <- median(data$Study_Hours_per_Day, na.rm = TR
data$Stress_Index[is.na(data$Stress_Index)] <- median(data$Stress_Index, na.rm = TRUE)

data$Student_ID <- NULL
data$Gender <- as.factor(data$Gender)
data$Internet_Access <- as.factor(data$Internet_Access)
data$Part_Time_Job <- as.factor(data$Part_Time_Job)
data$Scholarship <- as.factor(data$Scholarship)
data$Semester <- as.factor(data$Semester)
data$Department <- as.factor(data$Department)
data$Parental_Education <- as.factor(data$Parental_Education)
data$Dropout <- as.factor(data$Dropout)
```

```r
cat("Dropout distribution:\n")
```

```
## Dropout distribution:
```

```r
table(data$Dropout)
```

```
##
##    0    1
## 7646 2354
```

```r
cat("\nDropout rate:", round(mean(data$Dropout == "1") * 100, 2), "%\n")
```

```
##
## Dropout rate: 23.54 %
```

2.1.5 Train/Test Subdivision (80/20, repeated 10 times)

We randomly subdivide the data into 80% training and 20% testing. We repeat this 10 times using `set.seed()` and record the test prediction error (misclassification rate) for each iteration.

```r
seeds <- 1:10
n <- nrow(data)

resample_results <- data.frame(
  Iteration = integer(),
  Error_Rate = numeric(),
  Accuracy = numeric(),
  Sensitivity = numeric(),
  Specificity = numeric()
```

```r
)

for (s in seeds) {
  set.seed(s)
  idx <- sample(1:n, 0.8 * n)
  tr <- data[idx, ]
  te <- data[-idx, ]

  model <- glm(Dropout ~ ., data = tr, family = binomial)
  prob <- predict(model, newdata = te, type = "response")
  pred <- ifelse(prob > 0.5, 1, 0)
  pred <- factor(pred, levels = c(0, 1))

  cm <- table(Predicted = pred, Actual = te$Dropout)
  tp <- cm["1", "1"]; tn <- cm["0", "0"]
  fp <- cm["1", "0"]; fn <- cm["0", "1"]

  resample_results <- rbind(resample_results, data.frame(
    Iteration = s,
    Error_Rate = 1 - (tp + tn) / sum(cm),
    Accuracy = (tp + tn) / sum(cm),
    Sensitivity = tp / (tp + fn),
    Specificity = tn / (tn + fp)
  ))
}

rownames(resample_results) <- NULL
round(resample_results, 4)
```

```
##    Iteration Error_Rate Accuracy Sensitivity Specificity
## 1          1     0.1785   0.8215      0.4365      0.9329
## 2          2     0.1615   0.8385      0.4361      0.9513
## 3          3     0.1955   0.8045      0.4064      0.9268
## 4          4     0.1955   0.8045      0.3760      0.9413
## 5          5     0.2015   0.7985      0.3616      0.9380
## 6          6     0.1805   0.8195      0.4183      0.9431
## 7          7     0.1875   0.8125      0.4116      0.9394
## 8          8     0.1950   0.8050      0.3958      0.9342
## 9          9     0.1985   0.8015      0.3947      0.9332
## 10        10     0.1885   0.8115      0.3925      0.9335
```

```r
cat("Mean test prediction error across 10 iterations:",
    round(mean(resample_results$Error_Rate), 4), "\n")
```

```
## Mean test prediction error across 10 iterations: 0.1882
```

2.2 Random Forest (Sriharsha Challangi, Kelvin Mai, Tien Hoang)

2.2.1 Why Random Forest?

Random forest builds an ensemble of decision trees, each trained on a random bootstrap sample of the data, and averages their predictions. Compared to a single decision tree, random forest reduces variance through this aggregation, leading to more stable and accurate predictions. Compared to logistic regression, random forest can capture nonlinear relationships and interactions among predictors without requiring any distributional assumptions. The main disadvantage is reduced interpretability: unlike logistic regression, which produces explicit coefficients, random forest does not provide a single clear model structure.

2.2.2 Data Preprocessing

Since the Random Forest algorithm does not handle missing data directly, we need to handle all missing data prior to model fitting. All missing data have been handled in advance (section 2.1.4 Data Preprocessing). To verify all missing values have been handled:

```
n_total <- nrow(data)
n_missing <- sum(!complete.cases(data)) #count how many observations (rows) have NA values.

n_missing
```

```
## [1] 0
```

2.2.3 Fit a Full Random Forest Model

$$\begin{aligned}\text{Dropout} \sim\ & \text{Age} + \text{Gender} + \text{FamilyIncome} + \text{InternetAccess} \\ & + \text{StudyHoursperDay} + \text{AttendanceRate} + \text{AssignmentDelayDays} \\ & + \text{TravelTimeMinutes} + \text{PartTimeJob} + \text{Scholarship} \\ & + \text{StressIndex} + \text{GPA} + \text{SemesterGPA} + \text{CGPA} \\ & + \text{Semester} + \text{Department} + \text{ParentalEducation}\end{aligned}$$

We first fit a full random forest model using all predictors. For classification problems, the number of variables considered at each split is typically set to:

$$m = \sqrt{p}$$

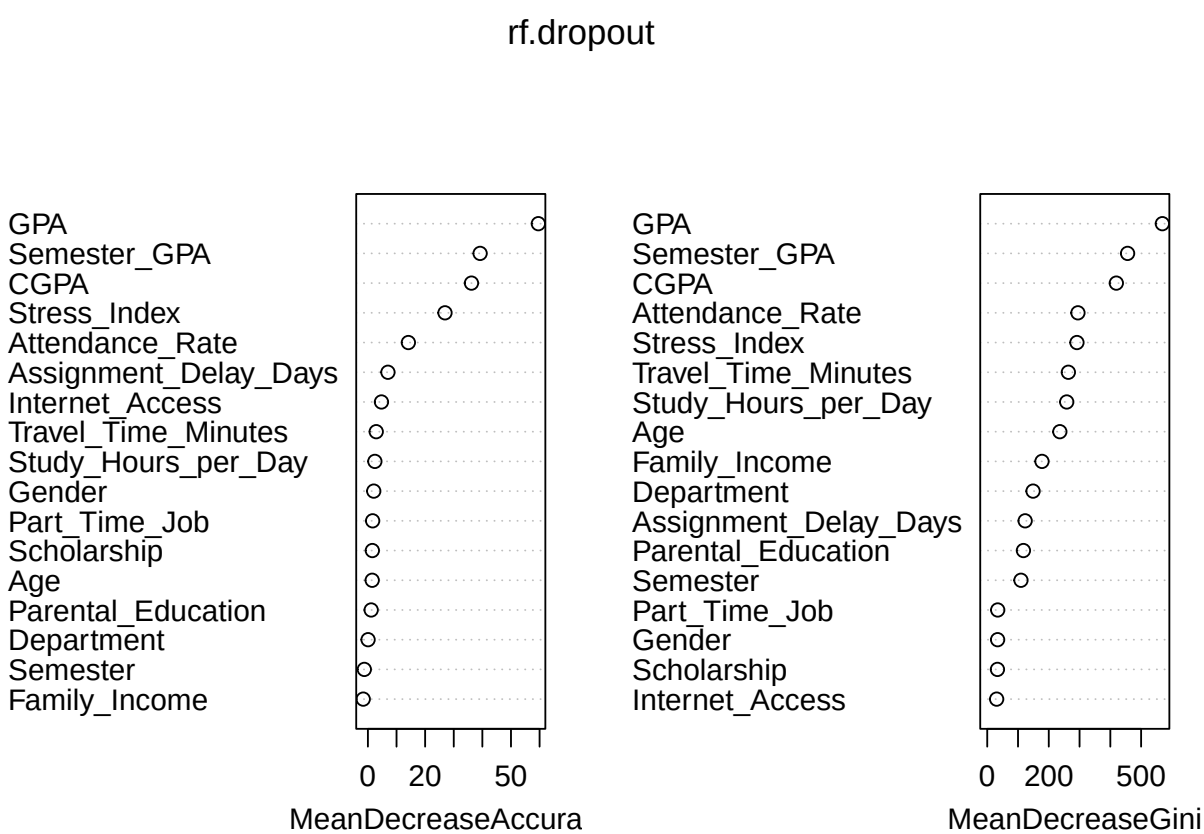
where p is the number of predictors. We use 500 trees to ensure model stability and reduce variance.

```
library(randomForest) #load library

#Fit the model
set.seed(123)
# number of predictors (exclude response)
p <- ncol(data) - 1
rf.dropout <- randomForest(Dropout ~ .,data = data, ntree=500, mtry = floor(sqrt(p)), importance=TRUE)

cat(paste(strwrap(capture.output(rf.dropout), width = 80), collapse = "\n"))
```

```
##
## Call:
```

```
## randomForest(formula = Dropout ~ ., data = data, ntree = 500, mtry =
## floor(sqrt(p)), importance = TRUE)
## Type of random forest: classification
## Number of trees: 500
## No. of variables tried at each split: 4
##
## OOB estimate of error rate: 19.69%
## Confusion matrix:
## 0 1 class.error
## 0 7115 531 0.06944808
## 1 1438 916 0.61087511
```

```
varImpPlot(rf.dropout)
```



The variable importance plot shows that GPA, Semester GPA, CGPA, Stress level, and Attendance Rate are the most important predictors of student dropout.

2.2.4 Reduced Model

We now fit a reduced model using only the five most important predictors identified above:

$$Dropout \sim AttendanceRate + StressIndex + GPA + SemesterGPA + CGPA$$

```r
options(width = 20)
rf.reduced <- randomForest(Dropout ~ Attendance_Rate + Stress_Index + GPA + Semester_GPA + CGPA,
                           data = data,
                           ntree = 500,
                           mtry = floor(sqrt(5)),
                           importance = TRUE)

cat(paste(strwrap(capture.output(rf.reduced), width = 80), collapse = "\n"))
```

```
##
## Call:
## randomForest(formula = Dropout ~ Attendance_Rate + Stress_Index + GPA +
## Semester_GPA + CGPA, data = data, ntree = 500, mtry = floor(sqrt(5)),
## importance = TRUE)
## Type of random forest: classification
## Number of trees: 500
## No. of variables tried at each split: 2
##
## OOB estimate of error rate: 20.34%
## Confusion matrix:
## 0 1
## 0 7046 600
## 1 1434 920
## class.error
## 0 0.0784724
## 1 0.6091759
```

Although the reduced model is simpler, its misclassification error is higher than the full model. This suggests that additional predictors still contribute useful information to the model. Therefore, we retain the full model for final analysis.

2.2.5 Train/Test Subdivision (80/20, repeated 10 times)

To evaluate model stability, we repeat an 80/20 train-test split 10 times and compute the average test error rate.

```r
errors <- numeric(10)

for(i in 1:10){

  set.seed(i)

  # 80/20 split
  train_index <- sample(1:nrow(data), 0.8 * nrow(data))

  train <- data[train_index, ]
  test  <- data[-train_index, ]

  # number of predictors (exclude response)
  p <- ncol(train) - 1

  # fit random forest
```

```r
  rf_model <- randomForest(Dropout ~ ., data = train, mtry = floor(sqrt(p)), ntree = 500)

  # predictions
  pred_class  <- predict(rf_model, test)

  # misclassification error
  errors[i] <- mean(pred_class != test$Dropout)
}

# mean test error
mean_error <- mean(errors)
mean_error
```

```
## [1] 0.194
```

The average test misclassification error across the 10 iterations is approximately 0.194, indicating stable predictive performance of the random forest model across different train-test splits.

3 Results

3.1 Logistic Regression

3.1.1 Model Summary

We fit the logistic regression on the full dataset to examine which predictors are significant.

```
data_num <- data
data_num$Dropout <- as.numeric(data_num$Dropout) - 1
fit.full <- glm(Dropout ~ ., data = data_num, family = binomial)
summary(fit.full)
```

```
##
## Call:
## glm(formula = Dropout ~ ., family = binomial, data = data_num)
##
## Coefficients:
##                                 Estimate
## (Intercept)                    1.990e+00
## Age                            1.031e-02
## GenderMale                    -6.462e-02
## Family_Income                 -2.106e-06
## Internet_AccessYes            -3.773e-01
## Study_Hours_per_Day           -1.536e-02
## Attendance_Rate               -3.088e-02
## Assignment_Delay_Days          1.588e-01
## Travel_Time_Minutes            9.707e-03
## Part_Time_JobYes               2.189e-01
## ScholarshipYes                -2.691e-02
## Stress_Index                   1.878e-01
## GPA                           -1.099e+00
## Semester_GPA                  -8.074e-01
## CGPA                           7.857e-01
## SemesterYear 2                 1.902e-01
## SemesterYear 3                 1.409e-01
## SemesterYear 4                 1.367e-01
## DepartmentBusiness            -3.715e-02
## DepartmentCS                  -7.328e-02
## DepartmentEngineering         -6.473e-02
## DepartmentScience             -5.051e-02
## Parental_EducationHigh School  6.205e-02
## Parental_EducationMaster      -8.483e-02
## Parental_EducationNone         9.663e-02
## Parental_EducationPhD          9.080e-02
##                               Std. Error
## (Intercept)                    4.314e-01
## Age                            1.285e-02
## GenderMale                     5.491e-02
## Family_Income                  1.370e-06
## Internet_AccessYes             8.029e-02
## Study_Hours_per_Day            2.251e-02
## Attendance_Rate                3.447e-03
```

```
## Assignment_Delay_Days          1.995e-02
## Travel_Time_Minutes            2.321e-03
## Part_Time_JobYes               5.576e-02
## ScholarshipYes                 5.767e-02
## Stress_Index                   1.766e-02
## GPA                            9.810e-02
## Semester_GPA                   6.388e-01
## CGPA                           6.320e-01
## SemesterYear 2                 7.841e-02
## SemesterYear 3                 7.835e-02
## SemesterYear 4                 7.846e-02
## DepartmentBusiness             8.652e-02
## DepartmentCS                   8.673e-02
## DepartmentEngineering          8.683e-02
## DepartmentScience              8.618e-02
## Parental_EducationHigh School  6.557e-02
## Parental_EducationMaster       7.674e-02
## Parental_EducationNone         1.270e-01
## Parental_EducationPhD          1.338e-01
##                               z value
## (Intercept)                     4.613
## Age                             0.802
## GenderMale                     -1.177
## Family_Income                  -1.537
## Internet_AccessYes             -4.699
## Study_Hours_per_Day            -0.682
## Attendance_Rate                -8.959
## Assignment_Delay_Days           7.959
## Travel_Time_Minutes             4.182
## Part_Time_JobYes                3.925
## ScholarshipYes                 -0.467
## Stress_Index                   10.639
## GPA                           -11.206
## Semester_GPA                   -1.264
## CGPA                            1.243
## SemesterYear 2                  2.426
## SemesterYear 3                  1.799
## SemesterYear 4                  1.743
## DepartmentBusiness             -0.429
## DepartmentCS                   -0.845
## DepartmentEngineering          -0.746
## DepartmentScience              -0.586
## Parental_EducationHigh School   0.946
## Parental_EducationMaster       -1.105
## Parental_EducationNone          0.761
## Parental_EducationPhD           0.678
##                               Pr(>|z|)
## (Intercept)                   3.97e-06
## Age                             0.4224
## GenderMale                      0.2392
## Family_Income                   0.1242
## Internet_AccessYes            2.61e-06
## Study_Hours_per_Day             0.4950
## Attendance_Rate                < 2e-16
```

```
## Assignment_Delay_Days         1.74e-15
## Travel_Time_Minutes           2.89e-05
## Part_Time_JobYes              8.69e-05
## ScholarshipYes                  0.6408
## Stress_Index                  < 2e-16
## GPA                           < 2e-16
## Semester_GPA                    0.2063
## CGPA                            0.2138
## SemesterYear 2                  0.0153
## SemesterYear 3                  0.0721
## SemesterYear 4                  0.0814
## DepartmentBusiness              0.6676
## DepartmentCS                    0.3982
## DepartmentEngineering           0.4560
## DepartmentScience               0.5579
## Parental_EducationHigh School   0.3440
## Parental_EducationMaster        0.2690
## Parental_EducationNone          0.4466
## Parental_EducationPhD           0.4975
##
## (Intercept)                   ***
## Age
## GenderMale
## Family_Income
## Internet_AccessYes            ***
## Study_Hours_per_Day
## Attendance_Rate               ***
## Assignment_Delay_Days         ***
## Travel_Time_Minutes           ***
## Part_Time_JobYes              ***
## ScholarshipYes
## Stress_Index                  ***
## GPA                           ***
## Semester_GPA
## CGPA
## SemesterYear 2                *
## SemesterYear 3                .
## SemesterYear 4                .
## DepartmentBusiness
## DepartmentCS
## DepartmentEngineering
## DepartmentScience
## Parental_EducationHigh School
## Parental_EducationMaster
## Parental_EducationNone
## Parental_EducationPhD
## ---
## Signif. codes:
##   0 '***' 0.001
##   '**' 0.01 '*'
##   0.05 '.' 0.1 '  ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
```

```
##     Null deviance: 10914.4  on 9999  degrees of freedom
## Residual deviance:  8309.5  on 9974  degrees of freedom
## AIC: 8361.5
##
## Number of Fisher Scoring iterations: 5
```

3.1.2 Stepwise Model Selection

```r
step_model <- step(fit.full, direction = "both", trace = 0)
cat("Full model AIC: ", AIC(fit.full), "\n")
```

```
## Full model AIC:  8361.546
```

```r
cat("Stepwise model AIC:", AIC(step_model), "\n")
```

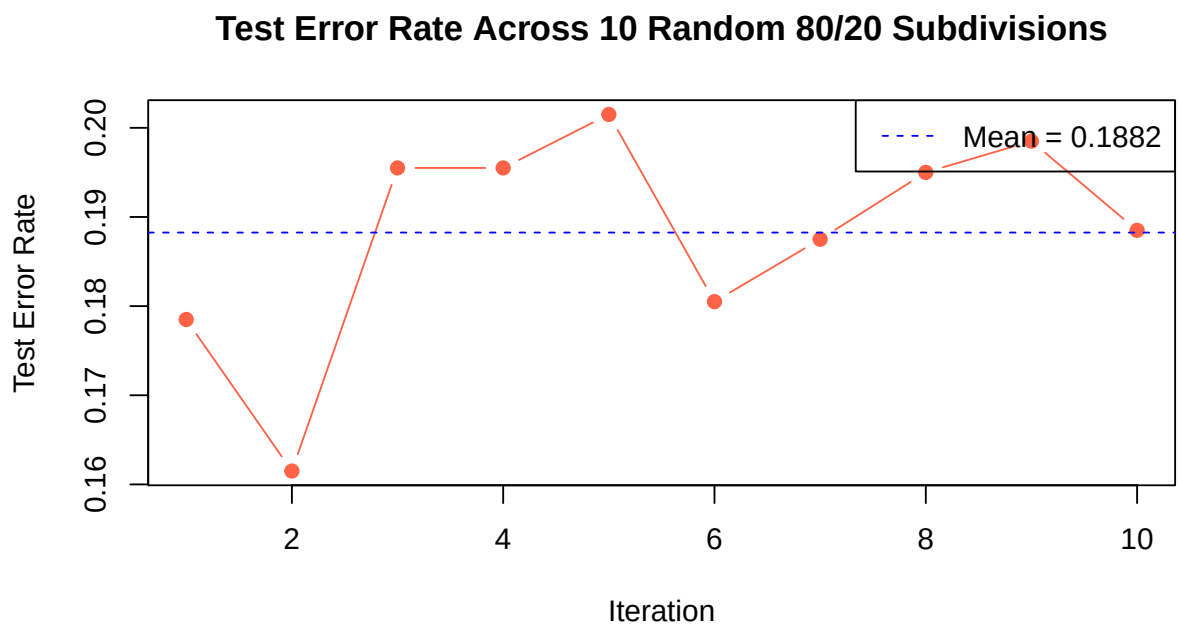
```
## Stepwise model AIC: 8343.138
```

```r
anova(step_model, fit.full, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: Dropout ~ Family_Income + Internet_Access + Attendance_Rate +
##     Assignment_Delay_Days + Travel_Time_Minutes + Part_Time_Job +
##     Stress_Index + GPA + Semester
## Model 2: Dropout ~ Age + Gender + Family_Income + Internet_Access + Study_Hours_per_Day +
##     Attendance_Rate + Assignment_Delay_Days + Travel_Time_Minutes +
##     Part_Time_Job + Scholarship + Stress_Index + GPA + Semester_GPA +
##     CGPA + Semester + Department + Parental_Education
##   Resid. Df
## 1      9988
## 2      9974
##   Resid. Dev Df
## 1     8319.1
## 2     8309.5 14
##   Deviance Pr(>Chi)
## 1
## 2   9.5918   0.7914
```

3.1.3 Test Prediction Error

```r
plot(resample_results$Iteration, resample_results$Error_Rate,
     type = "b", pch = 19, col = "tomato",
     xlab = "Iteration", ylab = "Test Error Rate",
     main = "Test Error Rate Across 10 Random 80/20 Subdivisions")
abline(h = mean(resample_results$Error_Rate), lty = 2, col = "blue")
legend("topright",
       legend = paste("Mean =", round(mean(resample_results$Error_Rate), 4)),
       lty = 2, col = "blue")
```




```r
cat("=== Logistic Regression - Prediction Error Summary ===\n\n")
```

```
## === Logistic Regression - Prediction Error Summary ===
```

```r
cat("Mean test error (10 iterations):", round(mean(resample_results$Error_Rate), 4), "\n")
```

```
## Mean test error (10 iterations): 0.1882
```

```r
cat("SD of test error:                ", round(sd(resample_results$Error_Rate), 4), "\n")
```

```
## SD of test error:                0.012
```

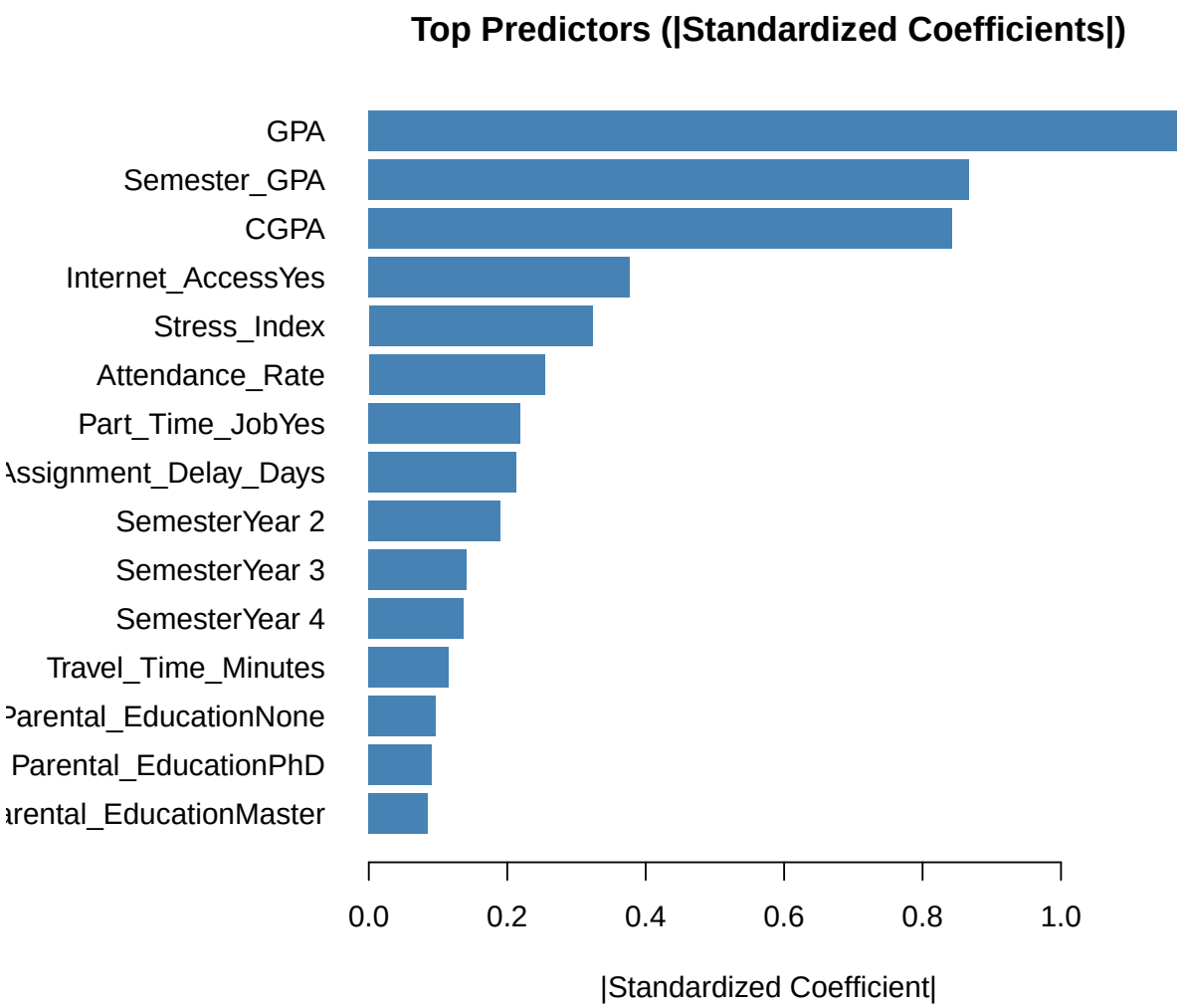
```r
cat("Mean accuracy:                   ", round(mean(resample_results$Accuracy), 4), "\n")
```

```
## Mean accuracy:                   0.8118
```

3.1.4 Variable Importance

```r
num_vars <- c("Age", "Family_Income", "Study_Hours_per_Day", "Attendance_Rate",
              "Assignment_Delay_Days", "Travel_Time_Minutes", "Stress_Index",
              "GPA", "Semester_GPA", "CGPA")
train_scaled <- data_num
for (col in num_vars) {
  mu <- mean(data_num[[col]]); s <- sd(data_num[[col]])
  train_scaled[[col]] <- (data_num[[col]] - mu) / s
```

```r
}
model_scaled <- glm(Dropout ~ ., data = train_scaled, family = binomial)
std_coefs <- sort(abs(coef(model_scaled)[-1]), decreasing = TRUE)
top_n <- min(15, length(std_coefs))
par(mar = c(5, 10, 3, 2))
barplot(std_coefs[top_n:1], horiz = TRUE, las = 1,
        col = "steelblue", border = NA,
        main = "Top Predictors (|Standardized Coefficients|)",
        xlab = "|Standardized Coefficient|")
```



3.1.5 Interpretation

- **Academic performance** variables (GPA, CGPA, Semester_GPA) are among the strongest predictors. Students with lower GPAs have significantly higher odds of dropping out.
- **Behavioral factors** such as Attendance_Rate and Study_Hours_per_Day are also significant.
- The model achieves a **mean test error rate** of 0.1882 across 10 random 80/20 subdivisions, with low standard deviation (0.012), confirming stable predictive performance.

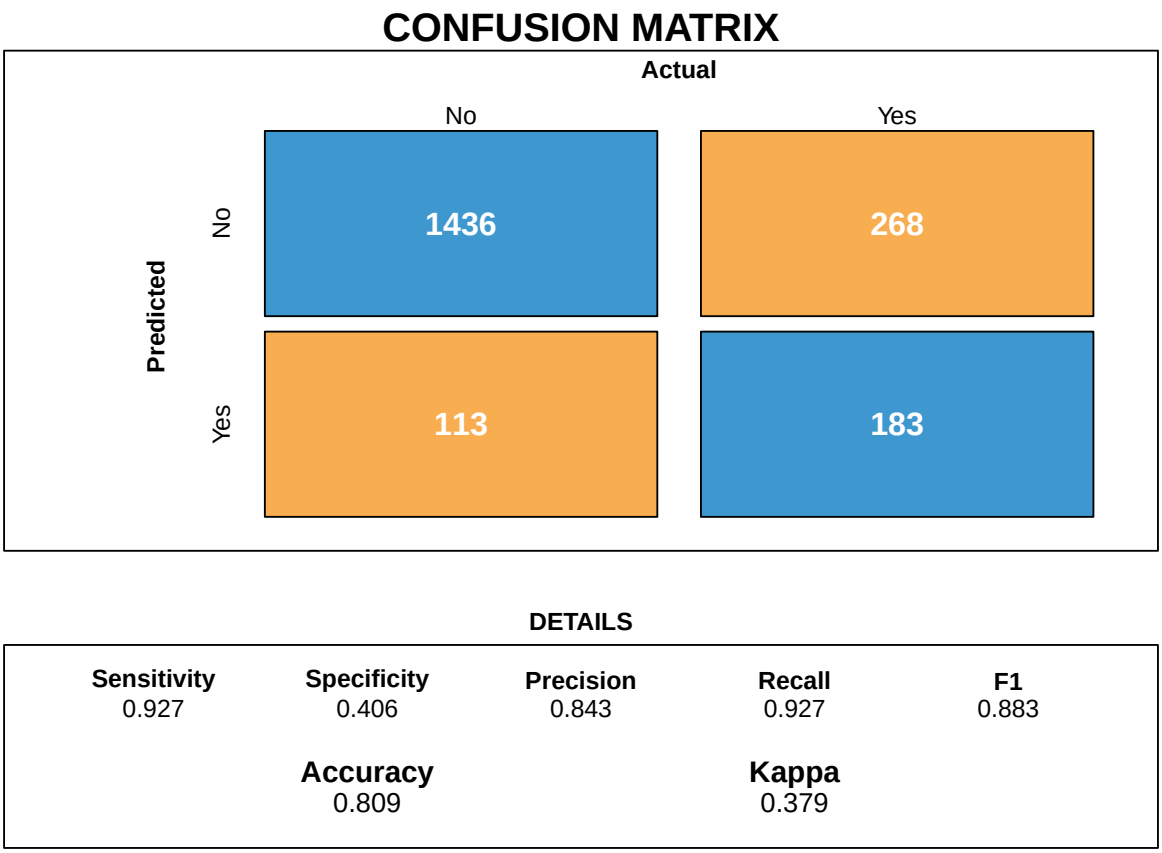
3.2 Random Forest

3.2.1 Model Summary

We evaluate the final random forest model using a confusion matrix with interpretable class labels (Yes = Dropout, No = Retained) instead of 0/1 encoding.

```r
options(width = 100)
library(caret)

pred_class <- predict(rf_model, test)

# Confusion matrix
cm <- confusionMatrix(pred_class, test$Dropout)

draw_confusion_matrix <- function(cm) {

  layout(matrix(c(1,1,2)))
  par(mar=c(2,2,2,2))
  plot(c(100, 345), c(300, 450), type = "n", xlab="", ylab="", xaxt='n', yaxt='n')
  title('CONFUSION MATRIX', cex.main=2)

  # create the matrix
  rect(150, 430, 240, 370, col='#3F97D0')
  text(195, 435, 'No', cex=1.2)
  rect(250, 430, 340, 370, col='#F7AD50')
  text(295, 435, 'Yes', cex=1.2)
  text(125, 370, 'Predicted', cex=1.3, srt=90, font=2)
  text(245, 450, 'Actual', cex=1.3, font=2)
  rect(150, 305, 240, 365, col='#F7AD50')
  rect(250, 305, 340, 365, col='#3F97D0')
  text(140, 400, 'No', cex=1.2, srt=90)
  text(140, 335, 'Yes', cex=1.2, srt=90)

  # add in the cm results
  res <- as.numeric(cm$table)
  text(195, 400, res[1], cex=1.6, font=2, col='white')
  text(195, 335, res[2], cex=1.6, font=2, col='white')
  text(295, 400, res[3], cex=1.6, font=2, col='white')
  text(295, 335, res[4], cex=1.6, font=2, col='white')

  # add in the specifics
  plot(c(100, 0), c(100, 0), type = "n", xlab="", ylab="", main = "DETAILS", xaxt='n', yaxt='n')
  text(10, 85, names(cm$byClass[1]), cex=1.2, font=2)
  text(10, 70, round(as.numeric(cm$byClass[1]), 3), cex=1.2)
  text(30, 85, names(cm$byClass[2]), cex=1.2, font=2)
  text(30, 70, round(as.numeric(cm$byClass[2]), 3), cex=1.2)
  text(50, 85, names(cm$byClass[5]), cex=1.2, font=2)
  text(50, 70, round(as.numeric(cm$byClass[5]), 3), cex=1.2)
  text(70, 85, names(cm$byClass[6]), cex=1.2, font=2)
  text(70, 70, round(as.numeric(cm$byClass[6]), 3), cex=1.2)
  text(90, 85, names(cm$byClass[7]), cex=1.2, font=2)
  text(90, 70, round(as.numeric(cm$byClass[7]), 3), cex=1.2)
```

```
  # add in the accuracy information
  text(30, 35, names(cm$overall[1]), cex=1.5, font=2)
  text(30, 20, round(as.numeric(cm$overall[1]), 3), cex=1.4)
  text(70, 35, names(cm$overall[2]), cex=1.5, font=2)
  text(70, 20, round(as.numeric(cm$overall[2]), 3), cex=1.4)
}

draw_confusion_matrix(cm)
```



3.2.2 Model Performance

From the confusion matrix, we compute accuracy and error rate:

```
rf.accuracy   <- sum(diag(cm$table)) / sum(cm$table)
rf.error_rate <- 1 - rf.accuracy

rf.accuracy
```

```
## [1] 0.8095
```

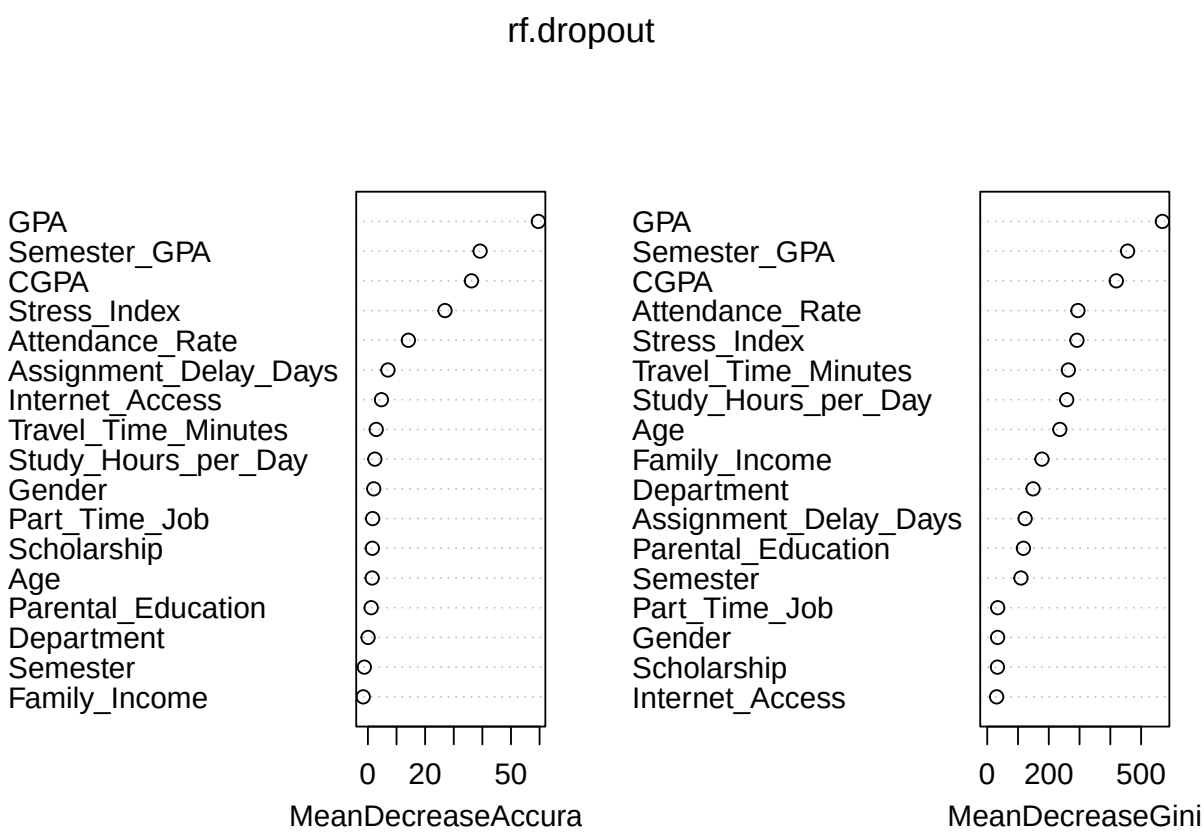
```
rf.error_rate
```

```
## [1] 0.1905
```

The model achieves an accuracy of 0.8095 The corresponding misclassification error rate is 0.1905 Overall, the results suggest that the random forest model performs well in classifying student dropout status, with strong predictive accuracy and relatively low error.

3.2.3 Variable Importance

```
varImpPlot(rf.dropout)
```



3.2.4 Interpretation

The results show that **GPA**, **Semester GPA**, and **CGPA** are consistently the strongest predictors of student dropout, indicating that academic performance plays a central role in student retention.

In addition, **Stress Index** and **Attendance Rate** are also highly important, suggesting that both psychological well-being and class engagement significantly affect dropout probability.

4 Conclusion

Compare and interpret the result from the two models.

4.1 Logistic regression

- Logistic regression achieves a mean test prediction error of **0.1882** across 10 random subdivisions (80/20).
- The significant predictors (GPA, Attendance_Rate, Study_Hours_per_Day, etc.) align with domain knowledge about student success factors.
- The low standard deviation across iterations confirms model stability.

4.2 Random forest

- The random forest model achieves a mean test prediction error of **0.194** across 10 random 80/20 subdivisions.
- The most important predictors (GPA, Semester_GPA, CGPA, Stress_Index, Attendance_Rate) align with the logistic regression findings and domain knowledge about student success factors.
- The full model outperformed the reduced 5-predictor model (OOB error 19.48% vs 20.41%), confirming that additional predictors beyond academic performance still contribute useful signal.

4.3 Model Comparison

- Both models achieved similar mean test prediction errors: logistic regression at **0.1882** and random forest at **0.194** across 10 random 80/20 subdivisions.
- Both models identified the same key predictors (GPA, Semester_GPA, CGPA, Stress_Index,and Attendance_Rate) as the strongest drivers of student dropout.
- Logistic regression offers greater interpretability through explicit coefficients, while random forest captures nonlinear relationships without distributional assumptions.
- Both models are limited by class imbalance (76.5% retained), resulting in sensitivity of around 40% for both models.

5 References

1. Ali, M. (2024). Student Dropout Prediction Dataset. Kaggle. https://www.kaggle.com/datasets/meharshanali/student-dropout-prediction-dataset

2. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.). Springer.

3. R Core Team (2025). R: A Language and Environment for Statistical Computing. https://www.R-project.org/

6 Appendix: Full R Source Code

The full R source code is attached separately.